



PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

November 16th, 2022

November 18th, 2022

DICTIONARIES

- ***Dictionary*** is similar to a ***List***, but more generic structure
- ***List*** can only have integer indices in a specified order
- ***Dictionary*** can have indices of any data type without any specified order

DICTIONARIES

- Indices are called ***keys***
- Data at these indices are called ***values***
- Therefore, ***Dictionary*** is a mapping between a set of ***keys*** and their associated ***values***
- Each ***key*** maps to a ***value***
- The association of a ***key*** and ***value*** is called a ***key-value pair*** or in short, ***item***
- For example, an English to Spanish dictionary that maps English words to Spanish:
 - Keys = English words (string)
 - Values = Spanish words (string)

DICTIONARIES

- Python has a built-in function called ***dict*** that creates a new dictionary with no items
 - `eng2sp = dict()`
`print(eng2sp)` → Output?
- The curly brackets { } represent an empty dictionary
- To add items, use square brackets:
 - `eng2sp['one'] = 'uno'`
`print(eng2sp)` → Output?
- This creates an item that maps the key “***one***” to value “***uno***”

DICTIONARIES

- Instantiate dictionaries like other variables:
 - `eng2sp = {'one' : 'uno', 'two' : 'dos', 'three' : 'tres'}`
- Printing a dictionary may result in a different order for different Operating Systems
 - The association remains unaffected by the order
 - For example,
 - `print(eng2sp)` → Output?
 - `print(eng2sp['two'])` → Output?
- If the key does not exist, it generates ***KeyError***

DICTIONARIES

- ***len*** function
 - Works on Dictionaries too
 - Counts the number of key-value pairs, i.e. items
 - For example,
 - `print(len(eng2sp))` → Output ?
- ***in*** operator
 - Works on Dictionaries too
 - Checking the ***key*** if it appears in the dictionary or not
 - Does not work for ***value*** part
 - For example,
 - `print('one' in eng2sp)` → Output?
 - `print('uno' in eng2sp)` → Output ?

DICTIONARIES

- ***values*** function
 - Checks for different values present in the dictionaries
 - Can use ***in*** operator for the values, only after extracting them in a list
 - For example,
 - `vals = eng2sp.values()`
 - `print('one' in vals)` → Output ?
 - `print('uno' in vals)` → Output ?

DICTIONARIES

- ***Implementation***

- A way of performing a computation
- Some can be better than others
- Implementation of Dictionary has an advantage over other structures
 - We don't need to specify in advance which elements will appear, and we can add only the ones that do
- For example,

- ```
def histograms(s):
 d = dict()
 for c in s:
 if c not in d:
 d[c] = 1
 else:
 d[c] += 1
 return d
```

- ```
h = histograms("brontosaurus")  
print(h)
```

 → Output?

DICTIONARIES

- ***get*** method
 - Takes a key and a default value as arguments
 - If key is in the dictionary, it returns the corresponding value
 - Otherwise, it returns back the default value
 - For example,
 - `h = histograms("a")`
`print(h)` → Output?
`print(h.get('a', 0))` → Output?
`print(h.get('b', 0))` → Output?

DICTIONARIES

- ***del*** method
 - Can be used to remove individual elements
 - Can delete the entire dictionary as well
 - For example,
 - `dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}`
`print(dict1)` → Output ?
`del dict1['Name']` # remove entry with key 'Name'
`print(dict1)` → Output ?
`del dict1` # delete entire dictionary
`print(dict1)` → Output ?

DICTIONARIES

- ***clear*** method
 - Can be used to clear or empty out the contents of the entire dictionary
 - For example,
 - `dict1 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}`
`dict1.clear()` `# remove all entries in dict1`
`print(dict1)` `→ Output ?`

LOOPS AND DICTIONARIES

- Using a for loop with Dictionaries applies to keys only
- It traverses the keys by default, and not their values
- For example,
 - `def print_hist(h):`
 `for c in h:`
 `print(c, h[c])` → Output ?
 - `h = histograms("parrots")`
 `print_hist(h)` → Output ?
- Prints the output in no particular/fixed order

LOOPS AND DICTIONARIES

- ***keys*** function:
 - A built-in Python function
 - Returns the keys of dictionary as a list
 - No pre-defined order
 - For example,
 - `k = h.keys()`
`print(k)` → Output ?

LISTS AND DICTIONARIES

- Lists can appear as values in a dictionary
- For example,
 - A dictionary that maps from letters to their frequency count
 - What about a dictionary that maps frequency count to the letters instead?
 - Invert the original dictionary
 - However, there might be several letters with the same frequency count
 - So, the values will become a **list** of letters instead of one value

LISTS AND DICTIONARIES

- ```
def invert_dict(d):
 inverse = dict()
 for k in d:
 val = d[k]
 if val not in inverse:
 inverse[val] = [k]
 else:
 inverse[val].append(k)
 return inverse
```
- ```
h = histograms("parrots")  
print(h)                                → Output ?  
inverse = invert_dict(h)  
print(inverse)                          → Output ?
```

LISTS AND DICTIONARIES

- Each time through the loop, *k* gets a key from *d* and *val* gets its value
- If *val* is not in *inverse*, create a new item, and assign it with a *singleton*
 - A *singleton* is a list containing a single element only
- List can be only values, and not keys
 - For example,
 - `t = [1, 2, 3]`
`d = dict()`
`d[t] = "oops"` → Output ?

LISTS AND DICTIONARIES

- Keys must be immutable, because having a mutable key may or may not give error by assigning two completely different keys to the same value
- This can become difficult to map back the value to its key
- Lists are mutable, so they cannot be used as keys
- Dictionaries are mutable, so cannot use them as keys either